A combinatory formulation of PTS-rk

October 22, 2026

Abstract

PTS-rk admits a complete Hilbert presentation by finitely many numerically indexed axiom schemes, by the supersorted-system theorem of Bunder and Dekkers. A finite basis of fixed closed typed combinators is nevertheless impossible: application cannot increase the rank of its function's type. We give rank-typed BCKW arithmetic, an explicit Hilbert catalogue and a first-order algebraic presentation using quotient inductive-inductive-recursive syntax. Checked soundness and extraction are distinguished from the remaining formal completeness and adequacy proofs. All expansions retain the original annotations and use beta conversion only. Elementary bounds control normalization and every intermediate term; fixing the input's universe labels gives finite syntax carriers and a total, typed Agda apply-and-normalize function. Independent certificate checking gives contextual soundness in graded finite carriers and external exclusion of a closed proof of Void. Finite Church encodings give explicit rank-bounded code-to-code reduction programs, separated from the proposed internal consistency proof. A ground-query Horn reduction gives a precise, weaker sense of reduction to a locally finitely satisfiable theory.

1 The calculus and its logical reading

Terms of PTS-rk are variables, universes U_i ($i \in \mathbb{N}$), products $\Pi x : A.B$, annotated abstractions $\lambda x : A.t$, and applications tu. Contexts are lists of typed variables. Its distinctive rules are

$$\Gamma \vdash U_i : U_0, \qquad \frac{\Gamma \vdash A : U_a \quad \Gamma, x : A \vdash B : U_b}{\Gamma \vdash \Pi x : A.B : U_{r(a,b)}}, \qquad r(a,b) = \max(a+1, b).$$

Typing and conversion are those of the original calculus: annotations are retained, with neither cumulativity nor eta conversion. In particular, $U_i : U_0$ is not the usual ascending universe rule $U_i : U_{i+1}$.

A logical judgment $\Gamma \vdash t : A$ reads: under assumptions Γ, t is a proof of the proposition represented by A. Types may also describe data; there is no separate primitive universe of propositions. A product $\Pi x : A.B(x)$ is universal quantification over inputs $x : A$; when the input and output are propositions and B does not depend on x, it is implication $A \to B$. Quantification over $A : U_a$ allows types and propositions themselves as parameters.

Application is dependent elimination:

$$\frac{\Gamma \vdash f : \Pi x : A.B(x) \quad \Gamma \vdash u : A}{\Gamma \vdash fu : B(u)}.$$

For $A \to B$, this is modus ponens, with fu recording the proofs used. Natural deduction discharges assumptions by abstraction; a Hilbert system encodes discharge through closed axioms and application. A derivation certifies the judgment, while its subject term is the object-level proof. Beta reduction computes within that proof without changing its type.

2 Rank-typed BCKW arithmetic

The standard basis separates composition B, exchange C, weakening K and duplication W; we retain identity I as an additional atom. Their types and computation rules are

	type	beta rule
B	$(Y \to Z) \to (X \to Y) \to X \to Z$	$Bfgx \longrightarrow_\beta^* f(gx)$
C	$(X \to Y \to Z) \to Y \to X \to Z$	$Cfxy \longrightarrow_\beta^* fyx$
K	$X \to Y \to X$	$Kxy \longrightarrow_\beta^* x$
W	$(X \to X \to Y) \to X \to Y$	$Wfx \longrightarrow_\beta^* fxx$
I	$X \to X$	$Ix \longrightarrow_\beta^* x.$

Each atom expands to the corresponding annotated lambda term. Type instantiations are implicit in these formulas, but retained in the checked syntax; arrow ranks are computed by $r(a, b) = \max(a + 1, b)$, not by an untyped interpretation. The B, C, K, I fragment is affine in its value arguments: their computation rules do not duplicate them; W does [7]. This does not require type parameters to occur affinely in annotations.

Fix $X = U_i$, put $E = X \to X$ and $\mathsf{Ch} = E \to E$, and define

$$H = C(BB(CB))B, \qquad \overline{0} = KI, \qquad \overline{n+1} = W(H\overline{n}).$$

Here $Hnfgx \longrightarrow_\beta^* f(ngx)$. Thus the first codes are

$$\overline{0} = KI, \qquad \overline{1} = W(H(KI)), \qquad \overline{2} = W(H(W(H(KI)))).$$

After expanding H without sharing, $\overline{n}$ contains exactly n occurrences of W and $7n + 2$ **combinator atoms in total**: H has six atoms, and each successor layer adds one W. The count treats I as one atom and does not count implicit type indices. For this family those indices have fixed shapes independent of n.

Successor, addition and multiplication are the following typed codes, with independent type instantiations at each occurrence of an atom:

$$\begin{aligned} \mathsf{succ} &= BWH && : \mathsf{Ch} \to \mathsf{Ch}, \\ \mathsf{add} &= B\big(B(BW)(BBC)\big)(BB) && : \mathsf{Ch} \to \mathsf{Ch} \to \mathsf{Ch}, \\ \mathsf{mul} &= B && : \mathsf{Ch} \to \mathsf{Ch} \to \mathsf{Ch}. \end{aligned}$$

Their atom counts are $8, 9, 1$, respectively. Applied to Church arguments, their expansions satisfy

$$\begin{aligned} \mathsf{succ}\ \overline{n}\ f\ x &\longrightarrow_\beta^* f^{n+1}x, \\ \mathsf{add}\ \overline{m}\ \overline{n}\ f\ x &\longrightarrow_\beta^* f^{m+n}x, \\ \mathsf{mul}\ \overline{m}\ \overline{n}\ f\ x &\longrightarrow_\beta^* f^{mn}x. \end{aligned}$$

Theorem 2.1 (Rank-typed BCKW arithmetic)**.** *For simple-type instances over U_i, every displayed code has its stated original PTS-rk type. The type* Ch *has rank* 2*, and the three operation types have rank* 3*. The exact counts, arithmetic laws, and forward reduction of $\overline{n}$ to the unapplied annotated normal form $\lambda f : E.\lambda x : X.f^n x$ are checked.*

This establishes the numeral fragment, not completeness for arbitrary dependent proofs. The BCKW QIIRT of Section 6 likewise checks as an independent signature. To replace the complete Hilbert catalogue one must additionally derive its dependent substitution schemes, with their literal annotations and types. That translation remains open.

3 Why a fixed finite typed basis is impossible

Write $\rho(t) = s$ when $t : T$ and $T : U_s$: this is the rank of the *type* of t. In a well-formed context it is unique, invariant under beta conversion between typable terms, and satisfies

$$\rho(fu) \leq \rho(f).$$

Every universe atom has type rank zero, since $U_i : U_0 : U_0$. For finitely many fixed closed typed atoms $b_1, \ldots, b_m$, set $R = \max(0, \rho(b_1), \ldots, \rho(b_m))$. Induction on application trees bounds every typable expansion by R, even when every U_i is available. The bound also holds for typable beta-convertible outputs. But define

$$\begin{aligned} A_0 &= U_0, \quad A_{s+1} = \Pi x : A_s.U_0, \\ v_0 &= U_0, \quad v_{s+1} = \lambda x : A_s.U_0. \end{aligned}$$

Then $A_s : U_s$ and $v_s : A_s$, so v_{R+1} cannot be represented.

Theorem 3.1 (No finite fixed typed basis)**.** *No finite list of closed originally typed PTS-rk terms is applicatively complete up to original beta conversion, even with all universe atoms available. The representing application tree must itself be originally typable.*

This excludes a fixed typed BCKW basis or a single fixed typed combinator, but not numerically indexed families with unbounded type ranks. Such families supply the economical presentation below.

4 Dependent logical schemes

Only the natural numbers in the subscripts are external indices. Every displayed type or family parameter is an ordinary term argument. Define

$$\begin{aligned} I_a &= \lambda A : U_a.\lambda x : A.x, \\ K_{a,b} &= \lambda A : U_a.\lambda B : U_b.\lambda x : A.\lambda y : B.x, \\ P_{a,b} &= \lambda A : U_a.\lambda B : (\Pi x : A.U_b).\Pi x : A.Bx. \end{aligned}$$

Their axiom types are

$$\begin{aligned} I_a &: \Pi A : U_a.\, A \to A, \\ K_{a,b} &: \Pi A : U_a.\Pi B : U_b.\, A \to B \to A, \\ P_{a,b} &: \Pi A : U_a.\Pi B : (\Pi x : A.U_b).\, U_{r(a,b)}. \end{aligned}$$

For dependent substitution, abbreviate the parameter telescope by

$$\Theta_{a,b,c} = \big(A : U_a,\ B : \Pi x : A.U_b,\ C : \Pi x : A.\Pi y : Bx.U_c\big).$$

Here $\lambda\Theta$ and $\Pi\Theta$ bind its three entries in order. The fourth scheme is

$$S_{a,b,c} = \lambda\Theta_{a,b,c}.\lambda f : (\Pi x : A.\Pi y : Bx.Cxy).\lambda g : (\Pi x : A.Bx).\lambda x : A.fx(gx),$$
$$S_{a,b,c} : \Pi\Theta_{a,b,c}.\Pi f : (\Pi x : A.\Pi y : Bx.Cxy).\Pi g : (\Pi x : A.Bx).\Pi x : A.Cx(gx).$$

Logically, I proves reflexive implication, K adds an unused hypothesis, and S combines two deductions using the same hypothesis. The family C allows the conclusion to depend on both that hypothesis and the intermediate proof gx. The combinator P forms the quantified proposition itself.

Theorem 4.1 (Sound core schemes)**.** *For every numerical index, the four displayed terms have exactly the displayed original PTS-rk types. Their fully applied beta laws are*

$$I_a\, A\, x \longrightarrow^*_\beta x, \qquad K_{a,b}\, A\, B\, x\, y \longrightarrow^*_\beta x,$$
$$S_{a,b,c}\, A\, B\, C\, f\, g\, x \longrightarrow^*_\beta fx(gx), \qquad P_{a,b}\, A\, B \longrightarrow^*_\beta \Pi x : A.Bx.$$

The ranks of their types are respectively $a+1$, $\max(a+1, b+1)$, $\max(a+2, b+2, c+1)$ *and* $a+2$.

The rank of the type of $P_{a,b}$ is independent of b because its result is a universe symbol, itself of type U_0.

5 A complete Hilbert presentation

Take the sorts $\{U_i : i \in \mathbb{N}\}$, the axioms $U_i : U_0$, and the four preceding schemes. Complete the scheme set $\mathcal{B}$ by the following operations of Bunder–Dekkers, Definition 7.3 [1]. They operate on schemes with sort parameters, before numerical instantiation.

1. **Types of schemes.** From $M : T$ in $\mathcal{B}$, with T not a sort, include $T' : U_s$ whenever the original calculus types this scheme. Here T' is T, or is obtained by replacing a repeated occurrence of a sort parameter in T by a fresh sort parameter.
2. **Expose a product.** Whenever $\mathcal{B}$ contains

   $$(\lambda x_1 : A_1 \cdots \lambda x_{k-1} : A_{k-1}.\Pi x_k : A_k.B) : (\Pi x_1 : A_1 \cdots \Pi x_{k-1} : A_{k-1}.U_s),$$

   also include the corresponding well-typed scheme

   $$(\lambda x_1 : A_1 \cdots \lambda x_k : A_k.B) : (\Pi x_1 : A_1 \cdots \Pi x_k : A_k.U_t).$$

Only the sort constraints needed for each new scheme's own typing are retained. They are constraints on fixed scheme shapes, not assumptions that the theorem being translated is already derivable. The resulting catalogue has 11 product, 7 identity, 17 constant and 30 substitution shapes [1, Section 7]. Specialize their sort constraints using $U_i : U_0$ and $r(a, b) = \max(a+1, b)$. Thus one family indexed by a tag from 65 shapes and a finite tuple of natural numbers suffices. The number 65 is an upper bound from this construction, not a minimality result.

The judgment $\vdash_H M : A$ has empty context. Its leaves are the universe axioms and instances of $\mathcal{B}$; its inference rules are

$$\frac{\vdash_H M : \Pi x : A.B \quad \vdash_H N : A}{\vdash_H MN : B[N/x]} \qquad \frac{\vdash_H M : A \quad \vdash_H B : U_s \quad A =_\beta B}{\vdash_H M : B},$$

$$\frac{\vdash_H M : A \quad M \longrightarrow_\beta N}{\vdash_H N : A} \qquad \frac{\vdash_H M : A \quad A \longrightarrow_\beta B}{\vdash_H M : B}.$$

These are application, conversion, subject reduction and type reduction. There is no abstraction or product-formation inference rule: their effect is recovered from the schemes. Annotated lambda and product syntax remains inside the schemes, types and reductions.

PTS-rk is *supersorted*: every constant has a sort and every pair of sorts has a product rule. On well-formed contexts, type correctness and supersortedness identify the repository's sorted-domain abstraction rule with the standard sorted-product rule [1, Theorem 7.2]. Hence Bunder–Dekkers, Theorem 9.4, gives

$$\boxed{\vdash_H M : A \quad \Longleftrightarrow \quad \vdash_{\mathrm{rk}} M : A}.$$

This is an external completeness theorem, not a Rocq theorem or axiom of this development. It supplies a Hilbert logical system: closed proofs of exactly the original closed judgments arise from fixed schematic logical axioms and the four rules above. Only its annotated, beta-only presentation is used here.

Theorem 5.1 (Hilbert soundness and proof extraction)**.** *For any catalogue whose original atom typings have been independently proved, a Hilbert derivation of $M : A$ yields a binder-free applicative code c such that*

$$\vdash_{\mathrm{rk}} M : A, \qquad \vdash_{\mathrm{rk}} E(c) : A, \qquad E(c) \longrightarrow_\beta^* M.$$

Here E expands catalogue atoms and preserves application.

The atom-typing premise belongs to the metatheorem, not the definition of a Hilbert derivation. Extraction retains a code through both reduction rules; its expansion recovers the subject by forward beta reduction, not merely conversion. For the literal 65-shape catalogue below, atom typing is discharged: soundness and extraction hold without an additional premise. The extracted code represents a proof through the original typing of $E(c)$; types still use original annotated syntax.

5.1 The literal catalogue and its completeness boundary

A literal catalogue can be described by the four core schemes, their types, and finite prefixes of their parameter telescopes. The types of the core schemes and their successive product exposures are included. For each prefix of length k listed below, put an arbitrary universe U_d at its end and replace the first e products by lambdas, for every $0 \le e \le k$:

Telescope	Prefix lengths	Universe-tail templates, in exposure order
P	$2; 3$	P_2, P_3, P_8; P_7, P_9, P_{10}, P_{11}
I	2	I_5, I_6, I_7
K	$2; 3; 4$	K_{12}–K_{14}; K_{15}–K_{18}; K_7–K_{11}
S	$3; 4; 5; 6$	S_9–S_{12}; S_{13}–S_{17}; S_{18}–S_{23}; S_{24}–S_{30}

Length zero gives a universe; length one gives $P_5 = \Pi A : U_a.U_d$ and $P_4 = \lambda A : U_a.U_d$. The S prefix of length two is the P prefix of length two. The length-three P telescope is $(A : U_a, B :$

$\Pi x : A.U_b, x : A$); its terminal body Bx supplies $P_6 = \lambda A : U_a.\lambda B : (\Pi x : A.U_b).\lambda x : A.Bx$. The other telescopes are the full binder lists of I, K, S above. All domain-sort slots and the final slot d are independent. The labels are local names, not an identification with the paper's numbering.

These templates give 11 product, 7 identity, 18 constant and 30 substitution shapes. Every instance has a proved original typing and exact type rank. Every declared type is literally another catalogue subject or a universe; its Hilbert type judgment and every leading-product exposure are derivable. The redundant constant shape $K_{13}(a, b, c) = \lambda A : U_a.\Pi B : U_b.U_c$ has the independently typed representation

$$K_{0,0}\, U_1\, U_a\, (P_5(b, c)) \longrightarrow_\beta^* \lambda A : U_a.\Pi B : U_b.U_c.$$

Removing it leaves 65 shapes and exactly the same Hilbert judgments. The remaining P_4, P_5 also have independently typed representations in the four-family core, with forward beta recovery.

Corollary 5.2 (Completeness of the literal catalogue)**.** *The 65-shape Hilbert system derives exactly the closed judgments of original PTS-rk. This mathematical result uses [1]; its completeness direction is not yet a Rocq theorem.*

To prove it, first retain all 66 templates. A declared type is a universe, a core type, or a pure-product prefix ending in a universe. Freshening a repeated sort occurrence in operation (1) therefore only renames an independent domain slot or frees the final slot; the table contains every well-typed instance. Operation (2) increments e, and all exposures are present. The original functional sort rules fix each declared type. Thus the 66 templates contain the schematic catalogue of Definition 7.3; Theorem 9.4 supplies completeness by inclusion. Soundness supplies the reverse implication, and the checked 65/66 equivalence removes K_{13}. The symbolic template-inclusion step is not formalized in Rocq.

The formalized support includes contextual start and empty-context equivalence, internal substitution, classification of the seven variable/application-ended heads, literal product-exposure compatibility and beta simulation, I, K, S abstraction and product assembly given the required family derivations, and the local type-preserving six-to-three S contraction with shared premises. General chain shortening with a sharing-aware measure, internal type correctness and unconditional deduction admissibility remain unformalized. The badges certify the supporting results, not the completeness direction.

5.2 A Metamath Zero specification

This Hilbert calculus has a first-order specification in Metamath Zero (MM0) [6]. The accompanying `CombinatoryPTS-rk.mm0` uses strict sorts for natural-number indices, raw de Bruijn terms, finite ranked telescopes and provable statements. Lambda annotations remain part of the term syntax; beta reduction retains capture-avoiding substitution. Auxiliary Horn rules compute ranks, shifts, substitutions and reductions. The graph `form e L D E M T` folds a ranked telescope `L` over a body/type pair `D:E`, exposing `e` initial lambdas and leaving products afterwards. Only fixed axiom rows feed Hilbert typing: they generate the 11 product, 7 identity, 17 constant and 30 substitution shapes, without an opaque catalogue lookup or original-typing oracle.

Its Hilbert inference layer is the following literal source excerpt:

```
term h: tm > tm > judgment;
axiom h_universe (i: nat): $ h (u i) (u z) $;
```

```
axiom h_scheme (M A: tm): $ ax M A $ > $ h M A $;
axiom h_apply (f x A B T: tm):
  $ h f (pi A B) $ > $ h x A $ > $ sub z x B T $ > $ h (app f x) T $;
axiom h_convert (M A B C: tm) (r: nat):
  $ h M A $ > $ h B (u r) $ > $ red A C $ > $ red B C $ > $ h M B $;
axiom h_subject (M N A: tm): $ h M A $ > $ red M N $ > $ h N A $;
axiom h_type (M A B: tm): $ h M A $ > $ red A B $ > $ h M B $;
```

Here an MM0 `axiom` declares a rule of this specified logical system; the arrow between dollar-delimited statements separates premises from conclusion. It is not a new assumption in the Rocq development. Read the rules by their least syntactic derivability relation, not as a claim that every first-order model has only the intended numbers or terms. Only the fixed closed templates and universe terms initialize Hilbert typing; application, conversion and reduction then preserve closedness. In `h_convert`, `C` is a common beta reduct of the two types; original confluence identifies this condition with beta conversion. An MM0 proof certifies such a derivation, while the PTS subject is the proof object it types. The specification and ten example derivations[1], including capture-avoiding substitution, pass the MM0 elaborator and reference checker. These checks do not establish the general Rocq correspondence or source completeness. The full source is included in Appendix A, directly from the same file; it is longer than one page.

6 A first-order algebraic presentation as QIIRT

A generalized algebraic theory (GAT) specifies dependent sorts, operations and equations; its initial algebra is syntax modulo those equations [3, 4]. Here we present the contextual BCKW(I) signature with a product former, separately from the full Hilbert catalogue. The natural-model presentation [5] keeps the context as an index and computes each term's type recursively. The result is a quotient inductive-inductive-recursive type (QIIRT): its sorts, paths and typing function are defined together. The following excerpts come directly from the checked Cubical Agda source, BCKW.agda[2].

```
data Con : Set                -- Contexts of typed assumptions.
data Sub : Con → Con → Set    -- Sub Γ Δ replaces Δ-variables in Γ.
data Ty : Con → Nat → Set     -- Ty Γ r: types A : U_r in Γ.
data Tm : Con → Set           -- Tm Γ: terms; tyOf computes their type.

RankedTy : Con → Set
RankedTy Γ = Σ Nat (Ty Γ)

tyOf : ∀ {Γ} → Tm Γ → RankedTy Γ
```

Here $\mathrm{Ty}(\Gamma, r)$ represents types classified by U_r. Thus $\mathrm{RankedTy}\,\Gamma = \sum_r \mathrm{Ty}(\Gamma, r)$ retains the sort of the type, and the fibre

$$\mathrm{TmAt}(\Gamma, r, A) = \sum_{t:\mathrm{Tm}\,\Gamma} (\mathrm{tyOf}\,t = (r, A))$$

recovers terms at a specified type. Equality constraints serve in place of a type index: this is the Ford transformation, not untyped syntax.

[1] `../../docs/direct-r-handoff/verification/combinatory-mm0-2026-10-22/CombinatoryPTS-rk-examples.mm1`

[2] `agda/BCKW.agda`

6.1 Operations and equations

The context operations are empty context and extension $\Gamma \triangleright A$. Substitutions have identity, composition, the unique substitution to the empty context, guarded pairing, and projections. Write their actions as $A[\sigma]$ and $t[\sigma]$. In addition to the category laws, impose

$$\begin{gathered} A[\mathrm{id}] = A, \qquad A[\sigma][\nu] = A[\sigma \circ \nu], \\ t[\mathrm{id}] = t, \qquad t[\sigma][\nu] = t[\sigma \circ \nu], \\ \mathrm{fst}(\sigma, t) = \sigma, \quad \mathrm{snd}(\sigma, t) = t, \quad (\mathrm{fst}\,\sigma, \mathrm{snd}\,\sigma) = \sigma, \\ (\sigma, t) \circ \nu = (\sigma \circ \nu, t[\nu]). \end{gathered}$$

Pairing requires $\mathrm{tyOf}\, t = (r, A[\sigma])$. The singleton $\langle t \rangle = (\mathrm{id}, t)$ uses the type-identity path; lifting a substitution through a binder uses the type-composition path. These structural guards are proved in the source. Comprehension eta is a law of substitutions, not function eta.

The logical operations are universe types $U_i : \mathrm{Ty}(\Gamma, 0)$, universe terms $\mathrm{univ}_i : \mathrm{Tm}\,\Gamma$, guarded decoding El_r, dependent products and application. Their types satisfy

$$\mathrm{tyOf}(\mathrm{univ}_i) = (0, U_0), \qquad \mathrm{El}_0(\mathrm{univ}_i) = U_i,$$
$$\frac{A : \mathrm{Ty}(\Gamma, a) \quad B : \mathrm{Ty}(\Gamma \triangleright A, b)}{\Pi AB : \mathrm{Ty}(\Gamma, \max(a+1, b))}.$$

A codomain is syntax in an extended context, not a meta-function on terms. Application is exported with the usual argument-type guard:

```
app : ∀ {Γ a b} (A : Ty Γ a) (B : Ty (ext Γ A) b) (f x : Tm Γ)
    → tyOf f ≡ (max (suc a) b , Pi A B)
    → tyOf x ≡ (a , A) → Tm Γ
app {a = a} A B f x pf px = app-guarded A B f x pf
  (px • λ i → a , sym (ty-id A) i)
```

The primitive guard uses $A[\mathrm{id}]$ to keep recursive definitions strictly positive; the displayed wrapper derives it from the identity law. Substitution is natural for universes, decoding and products, with $(\Pi AB)[\sigma] = \Pi(A[\sigma])(B[\sigma \uparrow_A])$. The closed atoms are $B_{a,b,c}, C_{a,b,c}, K_{a,b}, W_{a,b}, I_a$ and $P_{a,b}$. The first five explicitly quantify over their type arguments at the ranks indicated by the subscripts, then have the types of Section 2; P has the type of Section 4. Their literal telescopes are defined directly in the source. Typed application spines fix the saturation arities of the guarded B, C, K, W, I laws. The P law is $\mathrm{El}_{r(a,b)}(P_{a,b} A B) = \Pi x : \mathrm{El}_a(A).\mathrm{El}_b(Bx)$, with the displayed applications and decodings well typed. There is no abstraction constructor, function eta or unrestricted quotation $\mathrm{Ty}(\Gamma, r) \to \mathrm{TmAt}(\Gamma, 0, U_r)$.

For a term path $L = R$, recursion must also supply a path between its types. The guarded path constructors therefore carry $e : \mathrm{tyOf}\, L = \mathrm{tyOf}\, R$, with recursive clause $\mathrm{tyOf}(\mathrm{law}\ e\ i) = e(i)$. Only the specified laws generate paths, not arbitrary pairs of equally typed terms. All four sorts are sets: the term-truncation coherence guard is derived from the sethood of $\mathrm{RankedTy}\,\Gamma$. Structural contraction already follows from comprehension: in $\Gamma \triangleright A$, the substitution (id, q_A) maps into $\Gamma \triangleright A \triangleright A[p]$. Substituting along it shares one variable. This is distinct from compiling every dependent term into closed B, C, K, W, I atoms; ordinary contextual application itself permits shared variables.

This gives executable, intrinsically typed quotient syntax. The source derives the structural guards; general admissibility of the logical equations' coherence guards, interpretation in

original annotated PTS-rk, equality reflection and dependent combinatory completeness remain separate obligations.

7 Elementary normalization and finite syntax carriers

7.1 The rank parameter and the reduction strategy

Let $|t|$ count all constructors of the original annotated syntax, including annotations. Use Knuth's tetration notation $a \uparrow\uparrow 0 = 1$ and $a \uparrow\uparrow (h+1) = a^{a \uparrow\uparrow h}$, and put $q = \max(2, N)$. The relevant bound d bounds the type rank of **every redex head** in t, also inside annotations. It is the repository's `RedexBound`, not merely the outer type rank $\rho(t)$. Indeed the closed terms

$$t_a = (\lambda x : (\Pi A : U_a . A \to A) . U_0) I_a$$

all have type U_0 and 13 nodes, while their root redex has rank $a + 2$. This separates the two rank notions; it is not a lower bound on the cost of reducing these particular terms.

Theorem 7.1 (Normal form and intermediate-size bound)**.** *Suppose* Γ *is well formed,* $\Gamma \vdash t : A$*,* $|t| = N$*, and* `RedexBound` *holds at* d*. There is a deterministic beta-reduction sequence to a normal* t^* *with* $\Gamma \vdash t^* : A$*. Every intermediate term, including* t^**, has at most* $q \uparrow\uparrow (d+1)$ *nodes and retains redex-rank bound* d*. The documented abstract normalization cost is at most* $(N + 5d + 2) \uparrow\uparrow (2d + 2)$*.*

Use d passes of the existing greedy normalizer [8]. Within a pass, reduce the annotation or left child first, then the body or right child, then contract the root when possible. A pass lowers the maximal redex rank; sequentialization bounds every single-beta intermediate by $2^{|t|}$. Iterating this binary exponential d times gives a bound no larger than $q \uparrow\uparrow (d+1)$; the comparison is checked. Original subject reduction and redex-rank preservation supply the typing and rank invariants. For fixed d, these functions are elementary, with tower heights linear in d.

For any fixed finite set of atom shapes, including the indexed catalogue, expansion satisfies $|E(c)| \le C\ell$ when c has ℓ atoms, with a shape-dependent constant C. Thus $\max(2, C\ell) \uparrow\uparrow (d+1)$ bounds normalization of its expansion. For type-indexed atoms the bound requires fixed type shapes, as in the numeral family; arbitrary type-index payloads must also be charged. This does not yet bound the length of a normal **binder-free BCKW output**: an annotation-preserving compiler back to that syntax, with a size theorem, is still needed. Numerical labels count as one node here; their bit lengths must be charged in a machine-level complexity claim.

7.2 Finite normal families and dependent application

Rank and length alone do not make normal forms finite: the infinitely many distinct $U_i : U_0$ are all closed normal terms of size one and type rank zero. Fix instead a finite alphabet Σ of universe labels. For a fixed type A, the desired family of **syntax codes** is

$$\mathrm{NF}_\Sigma(\ell, r; A) = \{t \mid \mathrm{labels}(t) \subseteq \Sigma, |t| = \ell,\ t \text{ normal},\ \vdash t : A,\ \vdash A : U_r\}.$$

This is a proposition-valued fibre of finite syntax, not a collection counting arbitrary typing derivations. The checked Agda construction below uses genuine normal representatives and an injective finite encoding. Universe labels and de Bruijn variables range over finite alphabets; products and lambdas retain both their annotation and body.

Theorem 7.2 (Finite carriers)**.** *Beta reduction introduces no universe labels. All closed terms of at most s nodes over Σ, and hence all their normal subfamilies, have an explicit complete finite enumeration of length at most $(3s+|\Sigma|+4) \uparrow\uparrow 3$ (duplicates are harmless). Consequently, for a closed typed d-bounded input of size N, every state of the preceding strategy lies in a finite carrier of size at most*

$$(4N+4) \uparrow\uparrow (d+3), \qquad \Sigma = \mathrm{labels}(t).$$

For closed normal $f : \Pi x : A.B$ and $y : A$, with $\Pi x : A.B : U_r$, let $N = 1 + |f| + |y|$. Running r passes on fy returns a normal $z : B[y/x]$. Every intermediate has size at most $\max(2, N) \uparrow\uparrow (r+1)$ and lies in the finite raw carrier over $\mathrm{labels}(f) \cup \mathrm{labels}(y)$ of capacity at most $(4N+4) \uparrow\uparrow (r+3)$.

The last rank bound needs no internal rank inference: normal f, y have no internal redexes, so only the new root redex matters. Its rank is r. The algorithm can represent each term state by an index into the displayed finite list; the proofs check the underlying normalizer and its finite envelope, not an extracted finite-memory machine. They do not bound the storage of arbitrary typing witnesses. Nor is an externally supplied, possibly beta-expanded B bounded by $|f| + |y|$: count supplied type syntax and its rank budget separately if types themselves are to be normalized.

7.3 Inductive-inductive-recursive normal representatives

The quotient syntax of Section 6 identifies beta-equivalent terms of different lengths. Length must therefore be measured on explicit representatives. NormalForms.agda[3] defines normal syntax, a non-lambda-head family, and their raw interpretation together:

```
module _ (Label : Set) where
  data Nf (n : Nat) : Set
  data NonLamNf {n : Nat} : Nf n → Set
  erase : ∀ {n} → Nf n → Raw Label n

  data Nf n where
    nf-U   : Label → Nf n
    nf-var : Fin n → Nf n
    nf-pi nf-lam : Nf n → Nf (suc n) → Nf n
    nf-app : (f x : Nf n) → NonLamNf f → Nf n

  data NonLamNf {n} where
    non-lam : ∀ {f : Nf n} → NotLambda (erase f) → NonLamNf f

  erase (nf-U i) = U i
  erase (nf-var i) = var i
  erase (nf-pi A B) = pi (erase A) (erase B)
  erase (nf-lam A b) = lam (erase A) (erase b)
  erase (nf-app f x p) = app (erase f) (erase x)
```

The occurrence of `erase` in the `non-lam` constructor makes this a genuine inductive-inductive-recursive definition. Here `n` is context length, while `erase` retains every original annotation; it forgets only the normal-form presentation. `NotLambda` excludes precisely lambda heads. Thus application cannot introduce a beta redex, and all children, including annotations, are normal. Conversely, every normal raw term has such a representative. The head witnesses are propositions and `erase` is injective. agda (proof)

[3] `agda/NormalForms.agda`

Exact and bounded node counts give the fibres

```
SizedNf : (a n s : Nat) → Set
SizedNf a n s = Σ (Nf (Fin a) n) (λ t → nodes (erase (Fin a) t) ≤ s)

ExactNf : (a n N : Nat) → Set
ExactNf a n N = Σ (Nf (Fin a) n) (λ t → nodes (erase (Fin a) t) ≡ N)
```

An `AtMost A k` certificate is a function `A → Fin k` together with its injectivity proof. Depth-bounded raw syntax has a sum/product encoding equivalent to `Fin (capacity a n s)`; every node-bounded term encodes and decodes identically. The arithmetic proof bounds this capacity by the displayed tetration. No normalization theorem is assumed. agda (proof) agda (proof)

NativeTyping.agda[4] transcribes the original six typing rules and annotated beta equivalence, not a bidirectional checker. For a fixed closed type `A`, `NormalAt labels N d A t` asserts normality, exactly `N` nodes, original typing at `A`, and a sort rank of `A` at most `d`. Typing and rank certificates are propositionally truncated so that different derivations do not create additional inhabitants. The resulting typed family and its cardinal bound are checked directly:

```
-- labels names the fixed alphabet; A is an original closed annotated
↪  type.
-- Length is exactly N; d bounds the rank of A, not just redexes (there
↪  are none).
TypedNormal : ∀ {a} → (labels : Fin a → Nat) → (N d : Nat) → Term 0 → Set
TypedNormal {a} labels N d A = Σ (ExactNf a 0 N)
  (λ t → NormalAt labels N d A (erase (Fin a) (fst t)))

typed-normal-bound : ∀ {a} (labels : Fin a → Nat) (N d : Nat) (A : Term
↪  0)
  → a ≤ N → AtMost (TypedNormal labels N d A)
      (tetration (4 · N + 4) (d + 3))
typed-normal-bound {a} labels N d A p = subtype-finite
  (λ t → NormalAt labels N d A (erase (Fin a) (fst t)))
  (λ t → normalAt-isProp labels N d A (erase (Fin a) (fst t)))
  (closed-exact-normal-bound a d N p)
```

The constructor `pack-typed-normal` includes every originally typed normal term over the chosen alphabet. Here $a \le N$ is essential for the alphabet-independent numerical bound: take the labels occurring in an input of size `N`, not an arbitrarily large ambient alphabet. The dictionary `labels : Fin a → Nat` and type `A` are fixed parameters. Duplicate dictionary entries merely overcount representatives; they do not invalidate the bound. The parameter `d` bounds the type rank in this normal family; a normal term's redex bound alone would be vacuous. agda (proof)

7.4 Total typed apply-and-normalize

BoundedReduction.agda[5] implements capture-avoiding substitution, the greedy pass and its iteration on original annotated syntax. It proves forward beta reduction, decidable normality, and compatibility with replacing finite label indices by their original universe numbers. The size proof is unconditional: one pass has at most $2^{|t|}$ nodes, and every single-beta intermediate of d passes has at most $T_d(|t|)$, where $T_0(N) = N$ and $T_{d+1}(N) = 2^{T_d(N)}$. This seeded binary tower is bounded by $\max(2, N) \uparrow\uparrow (d+1)$. agda (proof) agda (proof)

In FiniteTrace.agda[6], a state is literally `State a n s = Fin (capacity a n s)`. Decoding recovers the annotated term; `state-bound` injects the closed state type, for $s = T_d(N)$, into

[4] agda/NativeTyping.agda
[5] agda/BoundedReduction.agda
[6] agda/FiniteTrace.agda

`Fin` of size $(4N + 4) \uparrow\uparrow (d + 3)$. The computed beta path has finite-indexed states and proposition-valued edge evidence:

```
data FiniteWalk (a n s : Nat) : State a n s → State a n s → Set where
  finished : ∀ {i} → FiniteWalk a n s i i
  move : ∀ {i j k} → IndexedStep {a} {n} {s} i j
    → FiniteWalk a n s j k → FiniteWalk a n s i k
```

Here `IndexedStep i j` is the propositional truncation of an original beta step between the decoded indices. `rounds-trace` computes this walk with the exact input and output endpoints; `walk-sound` proves forward reduction after decoding. Every state has the bound, not the type of arbitrarily long whole walks. agda (proof)

For two closed IIR normal forms, put `N = 1 + |f| + |y|`. FiniteApplication.agda[7] defines the successful result type and proves its entire cardinal bound, including proof-irrelevant reduction evidence:

```
-- The output is a bounded IIR normal form, not a raw term with an opaque
↪  tag.
-- Truncation counts represented terms, rather than their reduction
↪  proofs.
ApplicationResult : (d : Nat) (f y : Nf (Fin a) 0) → Set
ApplicationResult {a} d f y =
  Σ (SizedNf a 0 (tower d (inputSize f y)))
    (λ v → ∥ BoundedSteps (tower d (inputSize f y))
      (application f y) (erase (Fin a) (fst v)) ∥₁)

application-result-bound : (d : Nat) (f y : Nf (Fin a) 0)
  → a ≤ inputSize f y
  → AtMost (ApplicationResult d f y)
      (tetration (4 · inputSize f y + 4) (d + 3))
application-result-bound {a} d f y labelsBound =
  subtype-finite (λ v → ∥ BoundedSteps (tower d (inputSize f y))
      (application f y) (erase (Fin a) (fst v)) ∥₁)
    (λ _ → isPropPropTrunc)
    (closed-normal-bound a d (inputSize f y) labelsBound)
```

The raw function `applyWithinFinite` returns `Maybe ApplicationResult`: without typing premises, the selected number of passes need not normalize. Its successful result is an actual normal representative with a certified bounded forward reduction. Size cannot cause failure. agda (proof)

TypedApplication.agda[8] packages the original premises $f : \Pi x : X.Y$, $y : X$, and $\Pi x : X.Y : U_d$ in `TypedInputs`. The inputs f, y are IIR normal forms; no normalization or output-typing certificate is an input. `TypedResult` includes the exact original output typing at $Y[y/x]$, the bounded forward reduction, and the same cardinal bound. The executable interface is total:

```
-- Total application at the exact substituted type, with a bounded beta
↪  trace.
-- Rank descent and subject reduction are proved from the original rules.
applyTypedWithin : ∀ {a} (inputs : TypedInputs a) → TypedResult inputs
applyTypedWithin inputs = normal-result inputs , output-typed inputs
```

`normal-result` constructs the normal representative of `rounds d (app f y)`; `output-typed` certifies its original dependent type. Typing premises are propositionally truncated: they

[7] agda/FiniteApplication.agda
[8] agda/TypedApplication.agda

justify normality and output typing, but do not select the computed syntax. The entire result type injects into `Fin` of size $(4N + 4) \uparrow\uparrow (d + 3)$, with $N = 1 + |f| + |y|$. agda (proof)

The Agda proof uses the original six rules throughout. Binder-aware algebra gives weakening and typed substitution, including annotations. agda (proof) Parallel reduction proves confluence and uniqueness of normal forms. agda (proof) Typing inversion, context conversion and substitution prove type correctness and subject reduction. agda (proof) Type-rank uniqueness and substitution safety show that one greedy pass lowers a positive redex bound; bound zero implies normality. Hence d passes normalize, and normal arguments give the required application bound. agda (proof) Every single-beta intermediate also preserves the original rank bound. agda (proof) `normalize` packages these results for any originally typed, rank-bounded term, including its normal form, exact typing and bounded trace. It is idempotent and independent of the sufficient rank budget; two normalized inputs are beta-convertible exactly when their normal outputs are equal. agda (proof)

NativeRankCoverage.agda[9] computes a sufficient redex bound from every original typing derivation in a well-formed context, recursing through the annotated subject and using type-rank uniqueness. Thus `normalize-typed` requires neither a supplied rank bound nor a termination certificate.

The finite-indexed trace certifies the syntax-state bound. Its construction still evaluates unrestricted `Raw` internally; neither it nor the result bound asserts that every host type, proof object or runtime allocation is bounded. The following finite-table compilation makes the executable state interface explicit without claiming such a whole-runtime bound.

7.5 Church compilation at explicit ranks

Fix a nonempty label alphabet of size $a \le N$, and put $s = T_d(N)$ and $K = \mathrm{capacity}(a, 0, s)$, so $1 \le K \le (4N + 4) \uparrow\uparrow (d + 3)$. For every positive m and fixed rank r, define the original PTS terms

$$\begin{aligned} \mathsf{E}^r_m &= \Pi X : U_r.\ \Pi x_0 : X. \cdots \Pi x_{m-1} : X.\ X, \\ \mathsf{c}^{m,r}_i &= \lambda X : U_r.\ \lambda x_0 : X. \cdots \lambda x_{m-1} : X.\ x_i \qquad (0 \le i < m). \end{aligned}$$

Then $\mathsf{E}^r_m : U_{r+1}$ and $\mathsf{c}^{m,r}_i : \mathsf{E}^r_m$. Every inner arrow has rank $r + 1$; the outer product has rank $\max(1, r + 1) = r + 1$ because $U_r : U_0$. Both the type and its canonical selectors are normal and have exactly $2m + 3$ annotated nodes. These facts, including original typing for every index, are checked in ChurchFinite.agda[10]. agda (proof)

The finite injections already proved give the explicit encoding

$$\ulcorner t \urcorner_r = \mathsf{c}^{K,r}_{\iota(t)}, \qquad \iota(t) = \mathrm{encodeState}(t).$$

For the exact-length normal family one can use the smaller $K_N = \mathrm{capacity}(a, 0, N)$ instead. Typing/rank proofs select an external subfamily of these codes; they are not additional stored data. This encodes the represented elements of the Agda fibres, not their Cubical paths or an internal dependent induction principle.

For a finite table $F : \mathrm{Fin}(m) \to \mathrm{Fin}(n)$, define

[9] `agda/NativeRankCoverage.agda`
[10] `agda/ChurchFinite.agda`

$$\mathsf{Map}_r(F) = \lambda p : \mathsf{E}^r_m.\ \lambda X : U_r.\lambda y_0 : X.\cdots\lambda y_{n-1} : X.\ p\,X\,y_{F(0)}\cdots y_{F(m-1)},$$
$$\mathsf{Map}_r(F) : \mathsf{E}^r_m \to \mathsf{E}^r_n, \qquad (\mathsf{E}^r_m \to \mathsf{E}^r_n) : U_{r+2},$$
$$\mathsf{Map}_r(F)\,\mathsf{c}^{m,r}_i \longrightarrow^*_\beta \mathsf{c}^{n,r}_{F(i)}.$$

This uses the same answer type X throughout: it does not instantiate p at the higher-ranked type E^r_n, and requires no cumulativity. The original typing, displayed ranks and forward beta computation of selectors, unary maps and their finite iterations are checked directly against the native PTS rules.

For a binary table $F(i,j)$, abbreviate its rows by $R_i = q\,X\,y_{F(i,0)}\cdots y_{F(i,n-1)}$ and use

$$\mathsf{Map}^{(2)}_r(F) = \lambda p : \mathsf{E}^r_m.\lambda q : \mathsf{E}^r_n.\lambda X : U_r.$$
$$\lambda y_0 : X.\cdots\lambda y_{\ell-1} : X.\ p\,X\,R_0\cdots R_{m-1}.$$

Its type is $\mathsf{E}^r_m \to \mathsf{E}^r_n \to \mathsf{E}^r_\ell$, again of rank $r+2$, and it computes $F(i,j)$ on canonical inputs. Indeed each row has type X, and successive elimination of the two selectors chooses $y_{F(i,j)}$. This binary generalization is implemented and regression-tested in Agda; its all-index typing and computation theorems are not yet formalized. Consequently rank 1 suffices for the finite data and rank 2 for the operators, taking $r = 0$. The number of branches, hence program and type size, grows with the external bounds; this is not a uniform fixed-interface encoding of all elementary computations [8].

7.5.1 Compiling reduction passes

Define a total finite transition $g : \mathrm{Fin}(K) \to \mathrm{Fin}(K)$ by decoding a state, performing one greedy pass, and re-encoding when the result has at most s nodes; otherwise leave the state unchanged. Similarly let $b(i,j)$ encode the application of the decoded terms when it fits, and return i otherwise. These fallback branches make the tables total on the entire ambient carrier. They are never used on the intended bounded runs.

FiniteMachine.agda[11] implements these functions. Writing $t_{ij} = \mathrm{app}(\mathrm{decode}\,i, \mathrm{decode}\,j)$, it proves, without a typing premise,

$$|t| \le N \implies \mathrm{decode}(g^d(\iota(t))) = \mathrm{rounds}(d,t),$$
$$T_d(|t_{ij}|) \le s \implies \mathrm{decode}(g^d(b(i,j))) = \mathrm{rounds}(d,t_{ij}).$$

agda (proof)

FragmentMachineSoundness.agda[12] proves typed soundness of these same finite-state functions. `finite-normalize` decodes a normal form with the original type and rank bound; `finite-apply` does so at exactly $Y[y/x]$ for normal typed inputs. Both return forward reduction and bounded single-beta trace certificates. Their numerical budget premises exclude clipping, not assume normalization. agda (proof)

The original PTS programs are now explicit finite terms:

[11] `agda/FiniteMachine.agda`
[12] `agda/FragmentMachineSoundness.agda`

$$
\begin{aligned}
G &= \mathsf{Map}_r(g), & A &= \mathsf{Map}_r^{(2)}(b),\\
\mathsf{Norm}^r_{a,d,N} &= \lambda p : \mathsf{E}^r_K.\ G^d p & &: \mathsf{E}^r_K \to \mathsf{E}^r_K,\\
\mathsf{Apply}^r_{a,d,N} &= \lambda p : \mathsf{E}^r_K . \lambda q : \mathsf{E}^r_K.\ G^d(Apq) & &: \mathsf{E}^r_K \to \mathsf{E}^r_K \to \mathsf{E}^r_K.
\end{aligned}
$$

Here $G^d p$ is a metalevel abbreviation for exactly d nested applications, not an added recursion primitive. The compiler tabulates one pass and the application constructor, then emits these compositions; it does not fill a table with precomputed final normal forms. For a closed originally typed input t over the chosen alphabet, with $|t| \le N$ and RedexBound(ε, t, d), the decoded output is its normal form. For normal $f : \Pi x : X.Y$ and $y : X$, take d as the product's sort rank and $N = 1 + |f| + |y|$: the output decodes to a normal $z : Y[y/x]$ by the existing normalization and preservation theorems.

The following executable generators return literal original syntax (`Term 0`), not host functions standing in for PTS terms. Here `compileMap` and `compileMap2` implement the displayed tables, and `repeatMap` emits the d applications:

```
-- S bounds annotated source nodes; every target is a literal original
↪  PTS
-- term. The transition table computes ONE greedy pass, not normal
↪  answers.
passProgram : (a S r : Nat) → Term 0
passProgram a S r = compileMap r (greedy-state {a = a} {n = 0} {s = S})

applicationProgram : (a S r : Nat) → Term 0
applicationProgram a S r = compileMap2 r (application-state {a = a} {n =
↪  0} {s = S})

reductionProgram : (a d N r : Nat) → Term 0
reductionProgram a d N r = repeatMap r (capacity a 0 (tower d N)) d
  (passProgram a (tower d N) r)

applyReductionProgram : (a d N r : Nat) → Term 0
applyReductionProgram a d N r = lam E (lam (shift E)
  (app (shift (shift (reductionProgram a d N r)))
    (app (app (shift (shift (applicationProgram a S r)))
      (var (fsuc fzero))) (var fzero))))
  where
  S = tower d N
  E = choiceType r (capacity a 0 S)
```

This is code-to-code computation, distinct from the total typed Agda function above. It does not itself return an inhabitant of $Y[y/x]$ inside PTS or translate arbitrary Agda programs and quotient/truncation proofs. The generic native map proof and the Agda generators are separate formalizations, not a checked cross-language compiler-correctness bridge. Tabulation can be extremely large; no practical construction time or whole-machine memory bound is asserted. In particular, the source budget N does not bound the generated program or its execution: even one canonical state code has $2K + 3$ nodes.

7.6 Graded fragment soundness

For a fixed label dictionary, let $\mathcal{F}_s(A)$ contain closed IIR normal forms of at most s annotated nodes with propositional original typing at A. FragmentEmbeddings.agda[13] gives an injection

[13] `agda/FragmentEmbeddings.agda`

into `Fin (capacity a 0 s)`. Increasing the size budget gives injective embeddings satisfying identity and composition, preserving the decoded term and its typing. For these normal representatives,

$$\mathrm{decode}(u) =_\beta \mathrm{decode}(v) \quad \Longleftrightarrow \quad \mathrm{decode}(u) = \mathrm{decode}(v).$$

The right-hand side is literal equality of native annotated terms, not of finite codes when the dictionary repeats labels. Original type conversion acts on the typing fibre and commutes with the size embeddings. agda (proof)

Typed apply-and-normalize gives a graded application operation: for $f \in \mathcal{F}_s(\Pi x : X.Y)$ and $y \in \mathcal{F}_t(X)$ its output belongs to $\mathcal{F}_{T_d(1+s+t)}(Y[y/x])$, where $\Pi x : X.Y : U_d$. The API retains the alphabet premise $a \leq 1 + |f| + |y|$. The output decodes to the unique normal form of fy; subject reduction proves its exact type. Increasing budgets does not change the decoded result, and beta-convertible inputs give beta-convertible outputs. agda (proof) These are sound closed-term operations and coherent finite inclusions, not closure of one fixed carrier under every operation.

Alphabet inclusions preserve decoded terms and commute with normalization, size embeddings, conversion and application. Injective alphabet maps give injective carrier maps satisfying identity and composition. agda (proof) AlphabetCoverage.agda[14] constructs finite prefix alphabets for arbitrary annotated terms, contexts and valuations, with exact decoding. A prefix alphabet's size can exceed the number of syntax nodes; the general interpretation therefore includes alphabet size in its seed.

7.6.1 Contextual certificate soundness

An independent finite proof-tree grammar records all six typing rules and explicit beta instructions, including reduction inside annotations. `Accepted` recursively checks subtrees and local syntax equalities; `accepted?` decides this predicate without a native typing or normalization oracle. Acceptance implies original `Typ`, and every original derivation has an accepted certificate. This is checker adequacy, not a replacement typing judgment. agda (proof)

Context certificates check each declaration's sort; valuation certificates check every image against the substituted original context lookup. Their decidable acceptance predicates derive well-formedness and typed substitution. In particular, no native typing, well-formedness or normalization premise is hidden in the following input predicate: agda (proof)

```
record ModelInput (m n : Nat) : Set where
  field
    source : ContextCertificate m
    target : ContextCertificate n
    environment : ValuationCertificate m n
    derivation : Certificate m
open ModelInput public

InputAccepted : ∀ {m n} → ModelInput m n → Set
InputAccepted x = ContextAccepted (source x) × ContextAccepted (target x)
  × ValuationAccepted (certificate-context (source x))
      (certificate-context (target x)) (environment x)
  × Accepted (certificate-context (source x)) (derivation x)
```

[14] `agda/AlphabetCoverage.agda`

`model-soundness` interprets every accepted input, not merely closed terms or normal valuation images. It substitutes the environment into the subject and type, computes a common sufficient rank bound, and normalizes both:

```
ModelOutput : ∀ {m n} → ModelInput m n → Set
ModelOutput x = FiniteInterpretation to𝕟 (certificate-context (target x))
  (model-term-code x) (model-type-code x)

model-soundness : ∀ {m n} (x : ModelInput m n) → InputAccepted x →
↪  ModelOutput x
model-soundness x h = realize-typing to𝕟 (input-target-wf h)
  (model-term-code x) (model-type-code x)
  (subst2 (Typ (certificate-context (target x)))
    (sym (model-term-recovers x)) (sym (model-type-recovers x))
    ↪  (input-typing h))
```

`ModelOutput` contains their finite normal representatives and forward reductions, with original typing at the normalized type. Subject reduction also retains exactly the substituted source type. The algorithm chooses the alphabet and grades from the checked data; callers supply no guessed budget or desired output. `check-and-interpret` returns `nothing` only for rejected certificates, never because an accepted input failed to normalize. agda (proof)

The decoded interpretation respects conversion, is independent of typing witnesses and sufficient budgets, and commutes with valuation composition:

$$\mathrm{nf}\big(\mathrm{nf}(t[\sigma])[\tau]\big) = \mathrm{nf}\big(t[\tau \circ \sigma]\big).$$

Here valuations are arbitrary well-typed syntactic substitutions, not arbitrary set-valued assignments. NativeValuations.agda[15] proves identity, composition, context comprehension and substitution laws for the original annotated syntax; normalization preserves those equations. agda (proof)

Finiteness counts the context and type as data, not as unbounded external indices. FragmentUniverses.agda[16] bounds types, values and valuations; FragmentInterpretation.agda[17] constructs their jointly normalized representatives. Write $C_{a,n}(s) = \mathrm{capacity}(a, n, s)$ and $D_{a,n}(s) = \prod_{j<n} C_{a,j}(s)$. The joint context/type/value package $\mathcal{P}$ and context/substitution package $\mathcal{M}$ admit injections with bounds

$$|\mathcal{P}_{a,n,s,d}| \le D_{a,n}(s)\, C_{a,n}(s)^2,$$
$$|\mathcal{M}_{a,m,n,s}| \le D_{a,m}(s)\, D_{a,n}(s)\, C_{a,n}(s)^m.$$

Declarations retain their preceding scopes; all annotations and substitution images are counted, while semantic certificates are propositional fibres. Every typed valuation between well-formed contexts has such a package at computed bounds. Images may remain raw: independently normalizing the components of a dependent tuple is not assumed to preserve its exact type. These bounds concern the represented packages, not arbitrary proof trees or runtime allocations. agda (proof)

The interpretation is externally nontrivial. For $\mathsf{Void} = \Pi X : U_0.X$, canonical-form inversion rules out a normal inhabitant: a normal function must be a lambda, whose body would inhabit the arbitrary type variable X. In the context $X : U_0$, no normal neutral function or

[15] `agda/NativeValuations.agda`
[16] `agda/FragmentUniverses.agda`
[17] `agda/FragmentInterpretation.agda`

other normal constructor can do so. Total normalization gives `Void-empty` for every original closed typing derivation, hence `accepted-Void-rejected` and emptiness of every finite Void fibre. These are Agda metatheorems, not internal PTS consistency proofs. agda (proof)

7.7 Fragment interpretations and internal consistency

The result above is all-six-rule contextual soundness in a graded syntactic interpretation. Every fixed typed input has computed finite coverage, and larger grades cover more inputs. It is not yet a theorem that each stage is a **finite first-order model of a logical fragment**. That requires a total interpretation of the chosen signature on fixed finite domains, coherent dependent fibres, and all chosen axioms under every semantic valuation. Operations that increase grades do not automatically supply this closure. For example, iterating $A_0 = U_0$ and $A_{k+1} = \Pi x : U_0.A_k$ gives normal types of rank one for $k > 0$, using only label zero and $2k + 1$ nodes: a fixed size-bounded faithful type carrier cannot contain the whole family. This does not rule out nonfaithful finite models.

The independent checker and its external soundness provide a precise starting point for internal reflection. Their translation into PTS remains open: equations on canonical Church codes do not give a finite-case or dependent induction principle for arbitrary inhabitants of E^r_K. Compiler correctness must connect the encoded checker and interpretation to the specified certificate predicate, including its conversion witnesses.

The intended endpoint is an original PTS proof of a specified consistency sentence, not a metalevel assertion that no term of type Void exists. In the repository's consistency plan[18], this is $\vdash c : J(\mathcal{NAT}(C))$, where C says that the fixed predicate-only checker accepts no contradiction certificate, including all its trace witnesses. A promising route uses supplied tower resources to prove the checker's bounded soundness in elementary arithmetic, then Pakhomov's transfer for predicate-only Π_1 sentences [9]:

$$\mathsf{EA} \vdash C^{2\mathcal{S}} \quad \Longrightarrow \quad \mathsf{H} \vdash \mathcal{NAT}(C) \quad \Longrightarrow \quad \vdash c : J(\mathcal{NAT}(C)).$$

Here $\mathcal{S}$ is the superexponential cut; the last implication requires an actual interpretation J of the stated theory into PTS. The finite Church construction supplies computational ingredients, not those soundness and interpretation proofs. External results for every fixed budget are not one internal proof quantifying over all budgets. This distinction leaves the Pakhomov-style self-consistency objective plausible without identifying it with an unrestricted arithmetical consistency sentence. It also avoids inferring consistency from the unrelated all-true Horn models below.

7.8 A locally finitely satisfiable target

There is a precise ground-query reduction, distinct from a faithful finite model of syntax. Ground the Hilbert inference rules, including the auxiliary syntax, substitution and reduction graphs. Introduce one nullary predicate p_J for each closed judgment J, and, for every ground rule instance, add

$$J_1, \ldots, J_k \Rightarrow J \quad \mapsto \quad (p_{J_1} \wedge \cdots \wedge p_{J_k}) \Rightarrow p_J.$$

This is an effective positive Horn theory in a countably infinite recursive vocabulary. Finite conversion certificates can be enumerated; no decision procedure for untyped conversion is

[18] `../self_consistency/README.tex`

assumed. Grounding all the explicit MM0 graph rules is a concrete version of the same construction.

Theorem 7.3 (Ground derivability reduction)**.** *A judgment J has a finite source derivation exactly when the grounded Horn theory entails p_J. Every finite fragment of the target has a one-element first-order model.*

For preservation, induct on a derivation. For reflection, interpret p_J as "J has a finite derivation": this is the least model of the rules. For finite satisfiability, use a singleton domain and make every p_J true; indeed this satisfies the whole positive theory. The badge proves the generic Horn theorem; the effective Hilbert instantiation is the preceding mathematical construction, not a checked MM0/native adequacy bridge. Together with the external completeness theorem of Section 5, it yields the stated ground-query reduction for original closed judgments.

This conclusion holds independently of the normalization bounds. Those bounds additionally give finite representations of individual computations; they do not supply a finite-signature interpretation preserving arbitrary quantified formulas. Requiring finite models to contain faithful free natural-number syntax would already fail: injective successor with zero outside its image has no finite model. Thus neither a finite faithful syntax model nor that stronger first-order reduction is claimed.

8 The exact typed expansion quotient

Unlike the independent algebraic signature, this quotient is *relative to* native annotated terms Raw, applicative codes Code, expansion E, original typing Typ and beta equality Beta; Uraw(r) denotes U_r. The module Exact.agda[19] takes these data, not soundness or completeness proofs:

```
data TyExact where
  ty : {r : Nat} (A : Raw) → Typ A (Uraw r) → TyExact r
  tyBeta : {r : Nat} (A B : Raw)
         (dA : Typ A (Uraw r)) (dB : Typ B (Uraw r))
         → Beta A B → ty A dA ≡ ty B dB
  setTyExact : {r : Nat} → isSet (TyExact r)

rankedSetExact : isSet RankedTyExact
rankedSetExact = isSetΣ isSetℕ (λ _ → setTyExact)

data TmExact where
  tm : {r : Nat} (A : Raw) (dA : Typ A (Uraw r))
     (p : Code) → Typ (expand p) A → TmExact
  tmBeta : {r : Nat} (A B : Raw)
         (dA : Typ A (Uraw r)) (dB : Typ B (Uraw r))
         (p q : Code) (h : Typ (expand p) A) (k : Typ (expand q) B)
         → Beta A B → Beta (expand p) (expand q)
         → tm A dA p h ≡ tm B dB q k
  setTmExact : isSet TmExact

tyOfExact (tm {r} A dA p h) = r , ty A dA
tyOfExact (tmBeta {r} A B dA dB p q h k e v i)
  = r , tyBeta A B dA dB e i
tyOfExact (setTmExact t u p q i j)
  = rankedSetExact (tyOfExact t) (tyOfExact u)
      (cong tyOfExact p) (cong tyOfExact q) i j
```

[19] agda/Exact.agda

Both endpoints of a term path are originally typed; type equality is retained alongside equality of expansions. The common rank on each path matches uniqueness of native sorts. The fibre of tyOfExact recovers a specified ranked type without a term type index. Given native beta reflexivity, the beta constructors identify duplicate typing witnesses; set truncation makes equality witnesses irrelevant.

Ordinary inductive representatives with explicit typed beta relations are also defined in the same file. Their type-respect lemma is checked; the setoid laws follow when the supplied native beta relation is an equivalence. Deleting quotient paths without retaining those relations would instead give free syntax. Quotienting untyped codes first would be unsound as a typing representation: a beta class can contain untypable representatives.

All accompanying Agda modules are checked with Agda 2.8.0, Cubical v0.9 and the safe checker, without postulates, holes or disabled checks. This establishes well-formedness of the constructions, not an Agda-to-Rocq adequacy theorem or completeness of the BCKW signature.

9 Future work

The following three directions concern proposed systems, not changes to the original calculus or results established above.

1. **A quadratically checkable PTS-rk variant.** Apply the ideas of van den Berg and den Besten [10]: replace implicit definitional conversion by explicit propositional computation and transport witnesses. Develop an objective variant preserving the rank discipline, and prove deterministic worst-case $O(L^2)$ type checking, where L includes the context, term, type, numerical ranks and all annotations and witnesses. Their Theorem 7 proves quadratic checking for objective type theory; adapting it to PTS-rk is a separate obligation. This bounds checking, not proof search or normalization.

2. **A logic-free arithmetic of partial functions.** Develop a system analogous to logic-free primitive recursive arithmetic, with the following requirements.

   - Terms denote partial functions, rather than necessarily total functions. The basic judgment is extension, not equality:

     $$f \sqsubseteq g \quad \Longleftrightarrow \quad \forall \vec{x}, y \left(f(\vec{x}) \downarrow y \Rightarrow g(\vec{x}) \downarrow y \right).$$

     Thus the graph of f is included in the graph of g; symmetry is not an inference rule.

   - Every partial computable function must be expressible using a finite term-forming basis, in the spirit of finite bases for elementary recursive arithmetic. Computational completeness is a requirement on the intended standard interpretation, not on each finite model.

   - Use reflexivity, transitivity and substitution, together with a suitable induction rule. Substitution and induction must be sound for partiality and extension. Require a finite core of axioms and one infinite scheme exists_n, whose intended meaning is: either all numbers are equal, or there is an initial segment of at least n distinct natural numbers. This is a semantic specification to encode in the extension calculus, not an addition of unrestricted first-order logic.

- Require local finite satisfiability: for every finite numeral k, the finite core together with all exists_n for $n \leq k$ has a finite model validating the inference rules. Noncollapsed finite models realizing the retained initial segments are a stronger, useful goal; the collapse disjunct must not be mistaken for such a construction.
- Require deterministic worst-case $O(L^2)$ derivation checking, or better, in the full encoded derivation length, including numerals, substitutions and induction certificates. Checking must not decide semantic extension of arbitrary programs or execute potentially divergent computations.

3. **Interpretation of the type theory in the arithmetic.** Construct an effective, derivation-preserving interpretation of the variant in (1) inside the system in (2). Specify the translations of typing, explicit computation witnesses and contradiction, with sufficient reflection or model soundness to support the intended consistency application. This must go beyond the all-true ground-Horn reduction: finite satisfiability alone does not exclude the translated contradiction.

10 References

1. M. W. Bunder and W. J. M. Dekkers, *Are there Hilbert-style Pure Type Systems?* Logical Methods in Computer Science **4**(1:1), 2008, pp. 1–19. doi:10.2168/LMCS-4(1:1)2008[20]. Open-access paper[21].

2. R. B. Jones, *The Lambda-calculus, Combinatory Logic, and Type Systems*, 1998–1999. Conceptual overview[22].

3. A. Kaposi, A. Kovács and T. Altenkirch, *Constructing Quotient Inductive-Inductive Types*, Proceedings of the ACM on Programming Languages **3**(POPL), Article 2, 2019. doi:10.1145/3290315[23]. Author manuscript[24].

4. T. Altenkirch and A. Kaposi, *Type Theory in Type Theory using Quotient Inductive Types*, POPL 2016, pp. 18–29. doi:10.1145/2837614.2837638[25]. Conference paper[26]; expanded draft[27], Sections 3.1–3.3.

5. L.-T. Chen, F. Nordvall Forsberg and T.-C. Tsai, *Can We Formalise Type Theory Intrinsically without Any Compromise? A Case Study in Cubical Agda*, CPP 2026. doi:10.1145/3779031.3779090[28]. Author paper[29]; Agda artifact[30], inspected at commit `8db0830`. Sections 3–5 develop the natural-model QIIRT method.

6. M. Carneiro, *Metamath Zero: Designing a Theorem Prover Prover*, CICM 2020, LNCS **12236**, pp. 71–88. doi:10.1007/978-3-030-53518-6_5[31]. MM0 specification[32].

[20] https://doi.org/10.2168/LMCS-4(1:1)2008
[21] https://lmcs.episciences.org/839/pdf
[22] https://www.rbjones.com/rbjpub/logic/cl/cl017.htm#LSKICube
[23] https://doi.org/10.1145/3290315
[24] https://akaposi.github.io/finitaryqiit.pdf
[25] https://doi.org/10.1145/2837614.2837638
[26] https://akaposi.github.io/tt-in-tt.pdf
[27] https://akaposi.github.io/paper_ttintt.pdf
[28] https://doi.org/10.1145/3779031.3779090
[29] https://fredriknf.com/papers/tt-qiirt_cpp2026.pdf
[30] https://github.com/L-TChen/TTasQIIRT
[31] https://doi.org/10.1007/978-3-030-53518-6_5
[32] https://github.com/digama0/mm0/blob/0d414c0bfdaaeb7fea571895127abc1fa5a3d956/mm0.md

7. S. Broda and L. Damas, *On combinatory complete sets of proper combinators*, Journal of Functional Programming **7**(6), 1997, pp. 593–612. doi:10.1017/S0956796897002888[33].

8. Original PTS-rk manuscript[34], sections labelled `complexity`, `rep-bound`, `uniform-elementary` and `non-uniform-rep`, and the checked normalization development[35]. See `BoundedNormalization`, `Runtime`, `EstimatorChecks` and `AutomaticBound`.

9. F. Pakhomov, *A weak set theory that proves its own consistency*, arXiv:1907.00877v2, 2019. Paper[36]. Lemma 3.11 and Corollary 5.3 give the predicate-only transfer results; Appendix A relates the theory to Robinson's R.

10. B. van den Berg and M. den Besten, *Quadratic type checking for objective type theory*, arXiv:2102.00905, 2021. Paper[37]. Sections 2–3, especially Theorem 7.

[33] `https://doi.org/10.1017/S0956796897002888`
[34] `../../latex/PTS-rk.tex`
[35] `../NormalizationComplexity/README.md`
[36] `https://arxiv.org/pdf/1907.00877v2`
[37] `https://arxiv.org/abs/2102.00905`

A Complete MM0 source

The following listing is read from the same file as the excerpt in Section 5.2. The highlighting follows MM0's declarations, comments and dollar-delimited math strings.

```
-- PTS-rk: the 65-shape Hilbert theory (MM0 specification, not a consistency proof).
-- All judgments mean finite Horn derivability. Variables below are metasyntactic.
delimiter $ ( ) $;
strict sort nat;
strict sort tm;
strict sort tel;
strict provable sort judgment;
term z: nat;
term s: nat > nat;
term u: nat > tm;
term v: nat > tm;
term app: tm > tm > tm;
term pi: tm > tm > tm;
term lam: tm > tm > tm;
term nil: tel;
term cons: tm > nat > tel > tel;
def one: nat = $ s z $;
def two: nat = $ s one $;
def three: nat = $ s two $;
def four: nat = $ s three $;
def six: nat = $ s (s four) $;
def v0: tm = $ v z $;
def v1: tm = $ v one $;
def v2: tm = $ v two $;
def v3: tm = $ v three $;
def v4: tm = $ v four $;

-- Numerical graphs on constructor numerals.
term le: nat > nat > judgment;
term lt: nat > nat > judgment;
term max: nat > nat > nat > judgment;
axiom le0 (n: nat): $ le z n $;
axiom les (a b: nat): $ le a b $ > $ le (s a) (s b) $;
axiom lt0 (n: nat): $ lt z (s n) $;
axiom lts (a b: nat): $ lt a b $ > $ lt (s a) (s b) $;
axiom max0 (n: nat): $ max z n n $;
axiom maxz (n: nat): $ max n z n $;
axiom maxs (a b c: nat): $ max a b c $ > $ max (s a) (s b) (s c) $;

-- lift k M N increments free indices at least k; sub k X M N removes index k.
term lift: nat > tm > tm > judgment;
term sub: nat > tm > tm > tm > judgment;
axiom lu (k a: nat): $ lift k (u a) (u a) $;
axiom lvlo (k i: nat): $ lt i k $ > $ lift k (v i) (v i) $;
axiom lvhi (k i: nat): $ le k i $ > $ lift k (v i) (v (s i)) $;
axiom la (k: nat) (f g x y: tm):
  $ lift k f g $ > $ lift k x y $ > $ lift k (app f x) (app g y) $;
axiom lp (k: nat) (A C B D: tm):
  $ lift k A C $ > $ lift (s k) B D $ > $ lift k (pi A B) (pi C D) $;
axiom ll (k: nat) (A C B D: tm):
  $ lift k A C $ > $ lift (s k) B D $ > $ lift k (lam A B) (lam C D) $;
axiom su (k a: nat) (X: tm): $ sub k X (u a) (u a) $;
axiom svlo (k i: nat) (X: tm): $ lt i k $ > $ sub k X (v i) (v i) $;
axiom sveq (k: nat) (X: tm): $ sub k X (v k) X $;
axiom svhi (k i: nat) (X: tm): $ le k i $ > $ sub k X (v (s i)) (v i) $;
axiom sa (k: nat) (X f g x y: tm):
  $ sub k X f g $ > $ sub k X x y $ > $ sub k X (app f x) (app g y) $;
axiom sp (k: nat) (X Y A C B D: tm):
```

```
  $ sub k X A C $ > $ lift z X Y $ > $ sub (s k) Y B D $ >
  $ sub k X (pi A B) (pi C D) $;
axiom sl (k: nat) (X Y A C B D: tm):
  $ sub k X A C $ > $ lift z X Y $ > $ sub (s k) Y B D $ >
  $ sub k X (lam A B) (lam C D) $;

-- Original annotated beta reduction, reflexive-transitive and contextual; no eta.
term red: tm > tm > judgment;
axiom rr (M: tm): $ red M M $;
axiom rt (M N P: tm): $ red M N $ > $ red N P $ > $ red M P $;
axiom rb (A B X N: tm): $ sub z X B N $ > $ red (app (lam A B) X) N $;
axiom ra (f g x y: tm): $ red f g $ > $ red x y $ > $ red (app f x) (app g y) $;
axiom rp (A C B D: tm): $ red A C $ > $ red B D $ > $ red (pi A B) (pi C D) $;
axiom rl (A C B D: tm): $ red A C $ > $ red B D $ > $ red (lam A B) (lam C D) $;

-- Fixed ranked telescopes; annotations are in their successive binder scopes.
def t1 (a: nat): tel = $ cons (u a) z nil $;
def ti (a: nat): tel = $ cons (u a) z (cons v0 a nil) $;
def tk (a b: nat): tel =
  $ cons (u a) z (cons (u b) z (cons v1 a (cons v1 b nil))) $;
def tp (a b: nat): tel = $ cons (u a) z (cons (pi v0 (u b)) (s a) nil) $;
def tpx (a b: nat): tel =
  $ cons (u a) z (cons (pi v0 (u b)) (s a) (cons v1 a nil)) $;
def ts (a b c q r t: nat): tel =
  $ cons (u a) z (cons (pi v0 (u b)) (s a)
    (cons (pi v1 (pi (app v1 v0) (u c))) q
    (cons (pi v2 (pi (app v2 v0) (app (app v2 v1) v0))) r
    (cons (pi v3 (app v3 v0)) t (cons v4 a nil))))) $;
term stel: nat > nat > nat > tel > judgment;
axiom st (a b c q r t: nat):
  $ max (s a) (s b) q $ > $ max q c r $ > $ max (s a) b t $ >
  $ stel a b c (ts a b c q r t) $;
term prefix: nat > tel > tel > judgment;
axiom p0 (L: tel): $ prefix z L nil $;
axiom ps (k a: nat) (A: tm) (L R: tel):
  $ prefix k L R $ > $ prefix (s k) (cons A a L) (cons A a R) $;

-- form e L D E M T folds L over D:E, exposing exactly e initial lambdas.
-- This is a syntax graph, NOT a typing judgment. Only the fixed rows below use it.
term form: nat > tel > tm > tm > tm > tm > judgment;
axiom f0 (D E: tm): $ form z nil D E D E $;
axiom fp (a b c: nat) (A D E M: tm) (L: tel):
  $ form z L D E M (u b) $ > $ max (s a) b c $ >
  $ form z (cons A a L) D E (pi A M) (u c) $;
axiom fl (e a: nat) (A D E M T: tm) (L: tel):
  $ form e L D E M T $ > $ form (s e) (cons A a L) D E (lam A M) (pi A T) $;

-- Exactly 65 named shapes (some numerical instances coincide); K13 is omitted.
term ax: tm > tm > judgment;
axiom coreI (a: nat) (M T: tm):
  $ form two (ti a) v0 v1 M T $ > $ ax M T $;
axiom coreK (a b: nat) (M T: tm):
  $ form four (tk a b) v1 v3 M T $ > $ ax M T $;
axiom coreS (a b c: nat) (L: tel) (M T: tm):
  $ stel a b c L $ >
  $ form six L (app (app v2 v0) (app v1 v0)) (app (app v3 v0) (app v1 v0)) M T $ >
  $ ax M T $;
axiom coreP (a b r: nat) (M T: tm):
  $ max (s a) b r $ > $ form two (tp a b) (pi v1 (app v1 v0)) (u r) M T $ > $ ax M T $;
axiom typeI (a e: nat) (M T: tm):
  $ form e (ti a) v1 (u a) M T $ > $ ax M T $;
axiom typeK (a b e: nat) (M T: tm):
```

```
  $ form e (tk a b) v3 (u a) M T $ > $ ax M T $;
axiom typeS (a b c e: nat) (L: tel) (M T: tm):
  $ stel a b c L $ > $ form e L (app (app v3 v0) (app v1 v0)) (u c) M T $ > $ ax M T $;
axiom exposeP (a b: nat) (M T: tm):
  $ form three (tpx a b) (app v1 v0) (u b) M T $ > $ ax M T $;
axiom sortP1 (a d e: nat) (M T: tm):
  $ form e (t1 a) (u d) (u z) M T $ > $ ax M T $;
axiom sortP2 (a b d e: nat) (M T: tm):
  $ form e (tp a b) (u d) (u z) M T $ > $ ax M T $;
axiom sortP3 (a b d e: nat) (M T: tm):
  $ form e (tpx a b) (u d) (u z) M T $ > $ ax M T $;
axiom sortI (a d e: nat) (M T: tm):
  $ form e (ti a) (u d) (u z) M T $ > $ ax M T $;
axiom sortK2p (a b d: nat) (L: tel) (M T: tm):
  $ prefix two (tk a b) L $ > $ form z L (u d) (u z) M T $ > $ ax M T $;
axiom sortK2l (a b d: nat) (L: tel) (M T: tm):
  $ prefix two (tk a b) L $ > $ form two L (u d) (u z) M T $ > $ ax M T $;
axiom sortK3 (a b d e: nat) (L: tel) (M T: tm):
  $ prefix three (tk a b) L $ > $ form e L (u d) (u z) M T $ > $ ax M T $;
axiom sortK4 (a b d e: nat) (M T: tm):
  $ form e (tk a b) (u d) (u z) M T $ > $ ax M T $;
axiom sortS (a b c d e k: nat) (L R: tel) (M T: tm):
  $ stel a b c L $ > $ prefix (s (s (s k))) L R $ >
  $ form e R (u d) (u z) M T $ > $ ax M T $;

-- BEGINHILBERT
term h: tm > tm > judgment;
axiom h_universe (i: nat): $ h (u i) (u z) $;
axiom h_scheme (M A: tm): $ ax M A $ > $ h M A $;
axiom h_apply (f x A B T: tm):
  $ h f (pi A B) $ > $ h x A $ > $ sub z x B T $ > $ h (app f x) T $;
axiom h_convert (M A B C: tm) (r: nat):
  $ h M A $ > $ h B (u r) $ > $ red A C $ > $ red B C $ > $ h M B $;
axiom h_subject (M N A: tm): $ h M A $ > $ red M N $ > $ h N A $;
axiom h_type (M A B: tm): $ h M A $ > $ red A B $ > $ h M B $;
-- ENDHILBERT
```